

R Lecture: Writing Pipeline-Structured Functions

TS.To Duc Khanh

17/05/2026

In R data processing, a “pipeline-friendly” function typically has these characteristics:

- The first argument is the data (`.data`)
- Uses the `%>%` pipe or the native `|>` pipe
- Returns a `tibble/data.frame`
- Supports non-standard evaluation (NSE) with `{{ }}` or `.data`
- Compatible with `dplyr`, `purrr`, `tidyr`, `ggplot2`

The example below illustrates how to write a standard `tidyverse` function that:

- groups data,
- computes descriptive statistics,
- filters by a threshold,
- and returns a tibble.

```
library(dplyr)

summarise_by_group <- function(.data, group_var, target_var,
                              threshold = NULL, ...) {
  result <- .data |>
    group_by({{ group_var }}) |>
    summarise(
      mean = mean({{ target_var }}, ...),
      sd   = sd({{ target_var }}, ...),
      n    = n(),
      .groups = "drop"
    )

  if (!is.null(threshold)) {
    result <- result |>
      filter(mean >= threshold)
  }

  return(result)
}

iris |> summarise_by_group(Species, Sepal.Length)
```

```
## # A tibble: 3 x 4
##   Species    mean    sd     n
##   <fct>    <dbl> <dbl> <int>
## 1 setosa    5.01 0.352   50
## 2 versicolor 5.94 0.516   50
## 3 virginica 6.59 0.636   50
```

```
iris |> summarise_by_group(Species, Sepal.Length, threshold = 6)
```

```
## # A tibble: 1 x 4
##   Species    mean    sd     n
##   <fct>      <dbl> <dbl> <int>
## 1 virginica  6.59 0.636   50
```

In the pipeline function above, the arguments are interpreted as follows:

- `.data` is always the first argument, which allows the function to work well with pipes
- `{{ }}` enables variable name look-up following the tidy evaluation standard

Exercise 1: Write a pipeline function that standardizes a variable:

$$y = \frac{x - \bar{x}}{s},$$

where $\bar{x}$ and s are the sample mean and sample standard deviation of the variable to be standardized.

```
standardize <- function(.data, target_var, ...) {
  col_name <- paste0(deparse(substitute(target_var)), "_scaled")
  .data |>
    mutate(
      !!col_name := ({ target_var } - mean({ target_var }, na.rm = TRUE, ...)) /
                    sd({ target_var }, na.rm = TRUE, ...))
  ) |>
  as_tibble()
}

iris |> standardize(Sepal.Length) |>
  select(Species, Sepal.Length, Sepal.Length_scaled) |>
  head()
```

```
## # A tibble: 6 x 3
##   Species Sepal.Length Sepal.Length_scaled
##   <fct>      <dbl>          <dbl>
## 1 setosa      5.1          -0.898
## 2 setosa      4.9          -1.14
## 3 setosa      4.7          -1.38
## 4 setosa      4.6          -1.50
## 5 setosa      5           -1.02
## 6 setosa      5.4          -0.535
```

Exercise 2: Write a pipeline function that identifies outliers using the Hampel filter.

The Hampel filter detects outliers based on the **Median Absolute Deviation (MAD)**:

$$\text{MAD} = \text{median}(|x_i - \text{median}(x)|)$$

An observation x_i is flagged as an outlier if:

$$|x_i - \text{median}(x)| > k \times 1.4826 \times \text{MAD}$$

where $k = 3$ by default (the constant 1.4826 makes MAD consistent with the standard deviation under a normal distribution).

```
detect_outliers_hampel <- function(.data, target_var, k = 3, ...) {  
  .data |>  
    mutate(  
      .med      = median({{ target_var }}, na.rm = TRUE),  
      .mad      = mad({{ target_var }}, na.rm = TRUE),  
      is_outlier = abs({{ target_var }} - .med) > k * .mad  
    ) |>  
    select(-.med, -.mad) |>  
    as_tibble()  
}
```

```
iris |> detect_outliers_hampel(Sepal.Length) |>  
  filter(is_outlier == TRUE)
```

```
## # A tibble: 0 x 6  
## # i 6 variables: Sepal.Length <dbl>, Sepal.Width <dbl>, Petal.Length <dbl>,  
## #   Petal.Width <dbl>, Species <fct>, is_outlier <lgl>
```

```
iris |> detect_outliers_hampel(Sepal.Length) |>  
  count(Species, is_outlier)
```

```
## # A tibble: 3 x 3  
##   Species    is_outlier     n  
##   <fct>      <lgl>      <int>  
## 1 setosa    FALSE         50  
## 2 versicolor FALSE         50  
## 3 virginica FALSE         50
```

Note: Some principles to keep in mind:

- Do not use `attach()`
- Use `.data[[ ]]` when a variable name is passed as a string
- Prefer `tibble` over `data.frame`